Author: Ioanna Renta

Churn Analysis

Challenge #1: 2
Challenge #2: 4
Challenge #3 6
Challenge #4: 7
Challenge #5: 9

Challenge #1:

KNN is sensitive to the scale of features. Implement feature scaling:

- Use StandardScaler from sklearn.preprocessing
- Scale both training and test sets
- Compare performance before and after scaling
- Document the improvement

Answer:

Scaling matters for KNN with a concrete example.

Right now the features look like this:

tenure: 1, 34, 2, 45, 2 (range 0-72)
MonthlyCharges: 29.85, 56.95, 53.85 (range 18-118)
SeniorCitizen: 0, 0, 0, 0, 1 (range 0-1)

When KNN calculates distance between two customers it does something like:

distance = √((tenure1 - tenure2)² + (MonthlyCharges1 - MonthlyCharges2)² + ...)

A difference of 10 in tenure contributes the same as a difference of 10 in SeniorCitizen — but SeniorCitizen only goes from 0 to 1! So tenure and MonthlyCharges completely **dominate** the distance calculation just because they have larger numbers — not because they're more important.

StandardScaler fixes this by converting every feature to the same scale — mean of 0 and standard deviation of 1. So every feature contributes fairly to the distance.

The result proves:

```
================================================
FINDING BEST K VALUE
================================================
K= 1 | Accuracy: 0.7119 | Precision: 0.4570 | Recall: 0.4545
K= 3 | Accuracy: 0.7622 | Precision: 0.5657 | Recall: 0.4492
K= 5 | Accuracy: 0.7658 | Precision: 0.5791 | Recall: 0.4305
K= 7 | Accuracy: 0.7814 | Precision: 0.6260 | Recall: 0.4385
K= 9 | Accuracy: 0.7885 | Precision: 0.6532 | Recall: 0.4332
K=11 | Accuracy: 0.7871 | Precision: 0.6623 | Recall: 0.4037
K=15 | Accuracy: 0.7885 | Precision: 0.6696 | Recall: 0.4011

Best K by recall: K=1 with recall=0.4545

================================================
CHALLENGE 1: FEATURE SCALING
================================================

Before scaling (K=7):
  Accuracy:  0.7658
  Precision: 0.5791
  Recall:    0.4305

After scaling (K=7):
  Accuracy:  0.7601
  Precision: 0.5511
  Recall:    0.5187
```

Before scaling: Recall = 0.4305 (43%) **After scaling:** Recall = 0.5187 (52%)

That's a **9 percentage point jump in recall** just from scaling — the model now catches 52% of churners instead of 43%. This perfectly proves the theory we discussed — without scaling, the larger-scale features were dominating the distance calculation unfairly.

Notice the small tradeoffs:

- Accuracy dropped slightly (76.58% → 76.01%)
- Precision dropped slightly (57.91% → 55.11%)
- But **recall improved significantly** — which is what matters most for churn

Challenge #2:

Analyze which features are most important:

- Try removing features one by one and see how it affects performance
- Use correlation analysis to find features most related to churn
- Create visualizations showing feature importance

Answer:

What is feature importance?

Not all 30 features contribute equally to predicting churn. Some features are highly correlated with churn (like tenure, contract type, monthly charges) and some are almost irrelevant (like whether someone has a phone service).

Feature importance analysis answers: which features actually matter?

How do we measure it for this challenge?

We'll use correlation analysis — measuring how strongly each feature moves together with churn. Correlation goes from -1 to +1:

- +1 — perfectly positively correlated (feature goes up, churn goes up)
- -1 — perfectly negatively correlated (feature goes up, churn goes down)
- 0 — no relationship at all

For example we'd expect:

- tenure to be negatively correlated with churn (longer tenure = less churn)
- MonthlyCharges to be positively correlated with churn (higher charges = more churn)

Top 4 are the most meaningful:

```
==================================================
CHALLENGE 2: FEATURE IMPORTANCE ANALYSIS
==================================================

Top 10 features most correlated with Churn:
tenure                                  0.352229
InternetService_Fiber optic             0.308020
Contract_Two year                       0.302253
PaymentMethod_Electronic check          0.301919
StreamingMovies_No internet service     0.227890
TechSupport_No internet service         0.227890
DeviceProtection_No internet service    0.227890
OnlineBackup_No internet service        0.227890
OnlineSecurity_No internet service      0.227890
InternetService_No                      0.227890
```

1. **tenure (0.35)** — strongest predictor. Exactly as expected — newer customers churn more. The longer someone stays, the more loyal they become.
2. **InternetService_Fiber optic (0.31)** — Fiber optic customers churn significantly more. Likely because Fiber is more expensive and customers are more price-sensitive or have more alternatives.
3. **Contract_Two year (0.30)** — two year contract customers churn much less. Makes complete sense — they're locked in and committed.
4. **PaymentMethod_Electronic check (0.30)** — interesting! Electronic check users churn more. This might indicate less engaged or less satisfied customers — automatic payment methods (credit card, bank transfer) correlate with staying.

Key business insight from this: the company should focus on:

- Getting customers onto **longer contracts early**
- Monitoring **Fiber optic** customers closely
- Watching customers who pay by **electronic check**

Challenge #3

Try other sklearn classifiers:

- `LogisticRegression`
- `RandomForestClassifier`
- `SVC` (Support Vector Classifier)
- Compare their performance with KNN

Answer:

- **Logistic Regression** — despite the name it's a classification algorithm. It finds the best line (or boundary) that separates churners from non-churners. Fast, simple, very interpretable. Good baseline.
- **Random Forest** — builds many decision trees (hence "forest") and combines their votes. Each tree asks a series of yes/no questions about the features. Very powerful and handles imbalanced data better than KNN.
- **SVC (Support Vector Classifier)** — finds the optimal boundary between classes with the maximum margin. Very effective but slower to train on large datasets.

Logistic Regression wins on everything — highest accuracy, highest precision, highest recall. This is actually common with smaller, well-structured datasets. Simple algorithms often outperform complex ones when the data has clear linear patterns; best balance of catching churners (57% recall) while staying accurate (81%).

Challenge #4:

Implement additional evaluation techniques:

- ROC curve and AUC score
- Precision-Recall curve
- Cross-validation for more robust evaluation
- Learning curves to understand model behavior

Answer:

<u>ROC Curve (Receiver Operating Characteristic)</u>

So far our model makes a binary decision — churn or not churn. But internally it actually calculates a probability for each customer (e.g. 73% likely to churn). We then apply a threshold (usually 0.5) — above 0.5 = churn, below = no churn.

The ROC curve asks: what happens if we move that threshold?

- Lower threshold → catch more churners (higher recall) but more false alarms (lower precision)
- Higher threshold → fewer false alarms but miss more churners

The ROC curve plots this tradeoff visually. The AUC (Area Under Curve) summarises it as one number — closer to 1.0 is better, 0.5 means the model is no better than random guessing.

<u>Cross-validation</u>

Instead of splitting data once (80/20), cross-validation splits it 5 times in different ways and averages the results. This gives a more reliable estimate of real performance — less dependent on which particular rows ended up in train vs test.

```
==================================================
CHALLENGE 4: ADVANCED EVALUATION
==================================================

AUC Score: 0.8416
ROC curve saved!

Cross-validation (5 folds):
Accuracy: 0.8042 (+/- 0.0131)
Recall:   0.5485 (+/- 0.0233)
```

AUC Score: 0.8416 This is a good score — remember 0.5 = random guessing, 1.0 = perfect. At 0.84 the model has strong ability to distinguish churners from non-churners when you consider all possible thresholds. Much better than the raw recall of 57% suggested.

Cross-validation results:

- Accuracy: 0.8042 (+/- 0.0131)
- Recall: 0.5485 (+/- 0.0233)

Two things to notice:

- **The means are very close to our single split results** (80.70% accuracy, 56.68% recall) — this means our original 80/20 split was representative.
- **The standard deviations are tiny** — +/- 0.013 on accuracy and +/- 0.023 on recall. This means the model performs **very consistently** across all 5 different splits. A model that varies wildly across folds would be unreliable in production.

Overall picture of our Logistic Regression model:

- 80% accuracy
- 84% AUC
- 57% recall — consistently across folds
- Stable and reliable

Challenge #5:

- Calculate the cost of false negatives (missed churners)
- Calculate the cost of false positives (unnecessary retention efforts)
- Find the optimal threshold for churn prediction
- Create actionable recommendations for the business

Answer:

Right now we have these numbers from our confusion matrix:

- True Negatives (918) — correctly said "won't churn"
- False Positives (117) — said "won't churn" but they did ← missed churners
- False Negatives (213) — said "will churn" but they didn't ← unnecessary retention
- True Positives (161) — correctly said "will churn"

The business question is: what does each error cost?

- False Negative (missed churner) — a customer leaves. The company loses their monthly revenue forever. This is the expensive mistake.
- False Positive (unnecessary retention) — you offer a discount or retention package to someone who wasn't going to leave anyway. You spend money unnecessarily but don't lose the customer.

Typically in churn:

- Missing a churner costs ~12 months of revenue (they leave and you lose them)
- Unnecessary retention effort costs ~1 month of revenue (a discount offered)

So missing a churner is roughly 12x more expensive than a false alarm.

```
==================================================
CHALLENGE 5: BUSINESS IMPACT ANALYSIS
==================================================

Confusion Matrix breakdown:
True Negatives  (correct no churn): 925
False Positives (unnecessary retention): 110
False Negatives (missed churners): 162
True Positives  (caught churners): 212

Business Impact:
Cost of missed churners: $125,893.44
Cost of unnecessary retention: $7,123.60
Total cost with model: $133,017.04

Cost without any model: $290,642.88
Money saved by using model: $157,625.84

==================================================
OPTIMAL THRESHOLD ANALYSIS
==================================================
Threshold 0.3: Total cost=$88,462.16 | Caught 282 churners | Missed 92
Threshold 0.4: Total cost=$108,732.04 | Caught 250 churners | Missed 124
Threshold 0.5: Total cost=$133,017.04 | Caught 212 churners | Missed 162
Threshold 0.6: Total cost=$178,802.36 | Caught 149 churners | Missed 225
Threshold 0.7: Total cost=$241,684.32 | Caught 65 churners | Missed 309

Average monthly revenue per customer: $64.76
Cost per missed churner: $777.12
Cost per false alarm: $64.76
```

The model saves $157,625 compared to doing nothing — that's the headline number. Without the model you'd miss all 374 churners costing $290,642. With the model you only miss 162, costing $133,017.

Threshold	Total Cost	Caught	Missed
0.3	**$88,462**	282	92
0.4	$108,732	250	124
0.5	$133,017	212	162
0.6	$178,802	149	225
0.7	$241,684	65	309

Threshold 0.3 wins clearly — lowest total cost at $88,462. By lowering the threshold you're saying *"flag someone as a potential churner even if we're only 30% confident"* — and because missing a churner is 12x more expensive than a false alarm, being aggressive pays off.

The key business recommendation from this:

Set the model threshold to **0.3 instead of the default 0.5**. This would save an additional **$44,555** compared to using the default threshold.